4.0 Iterador While & Listas

Operador while y listas

Programación I
Ingeniería Electrónica y Telecomunicaciones

Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Rio Negro

Entorno

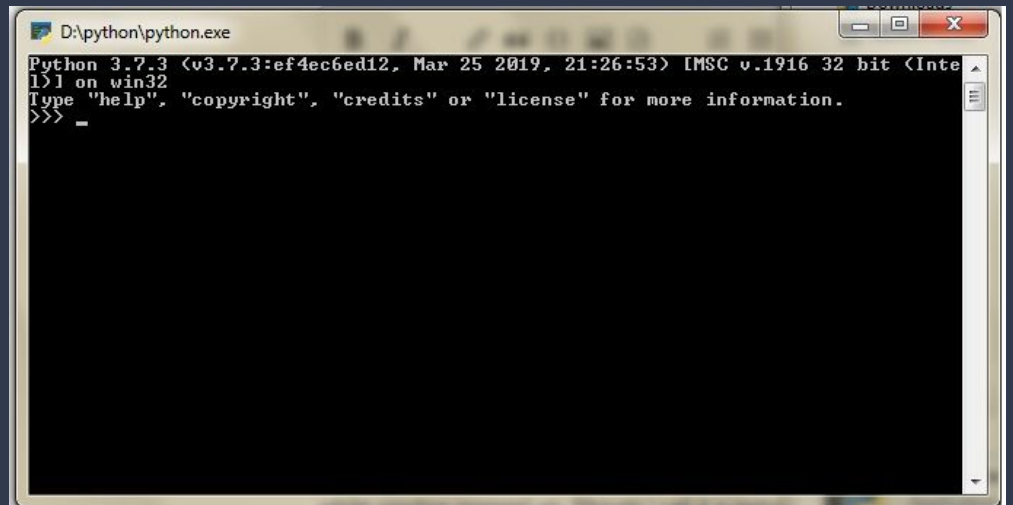
Repasando el enterno: Interprete interactivo vs IDE VSCode

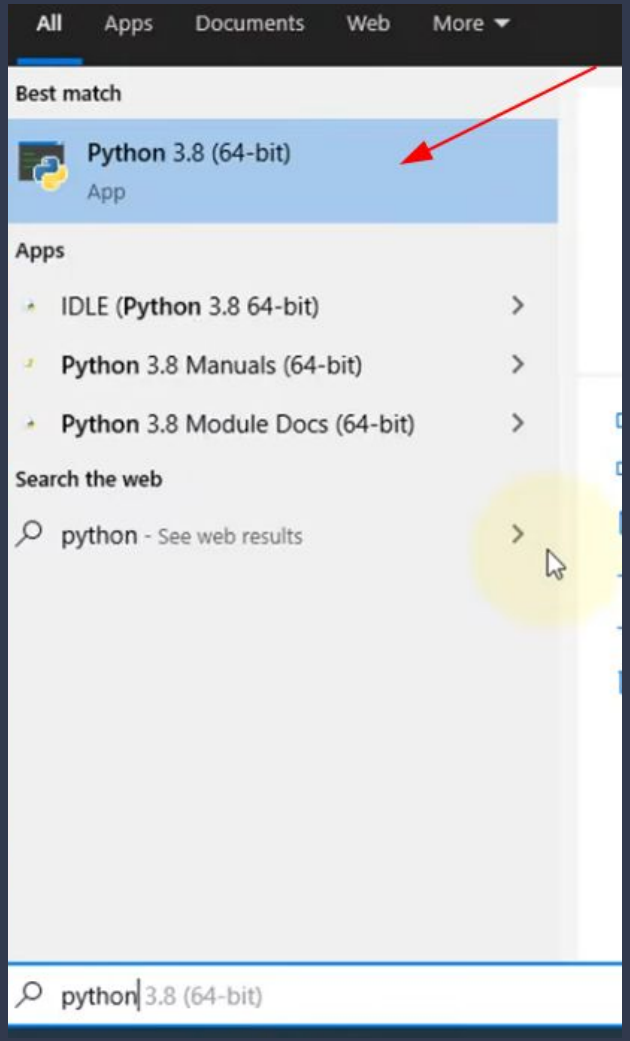
Repaso del entorno

Intérprete interactivo: nos permite hacer pruebas pero no es óptimo para escribir programas.

Llegamos a él con los siguientes pasos:

1. Inicio de windows
2. Buscar python
3. Abrir python.exe





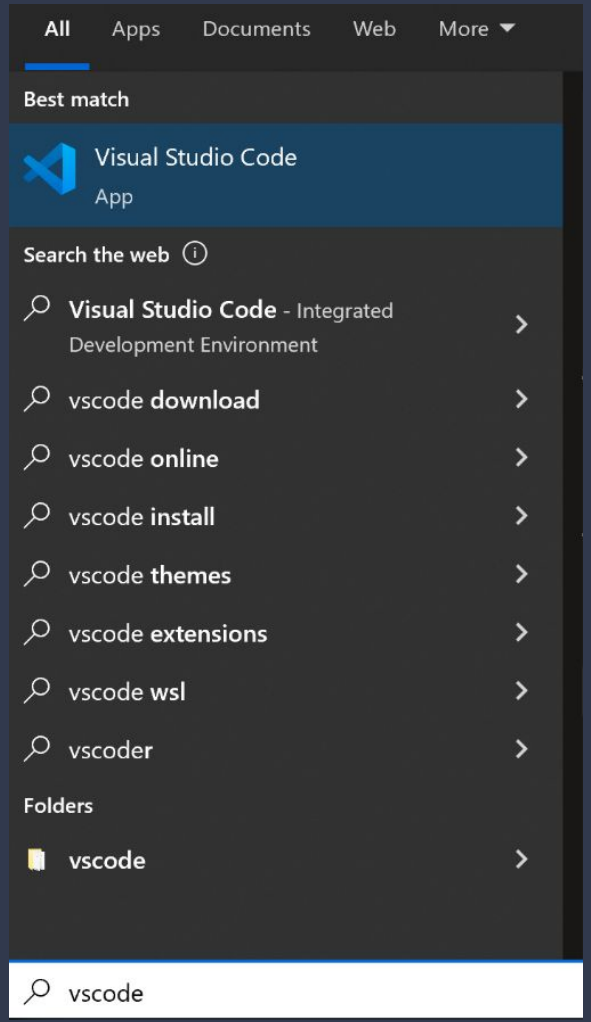
Repaso del entorno

IDE - VSCode: El ide será la herramienta preferencial para escribir programas, en nuestro caso utilizaremos Visual Studio Code.

IDE = Entorno de Desarrollo Integrado

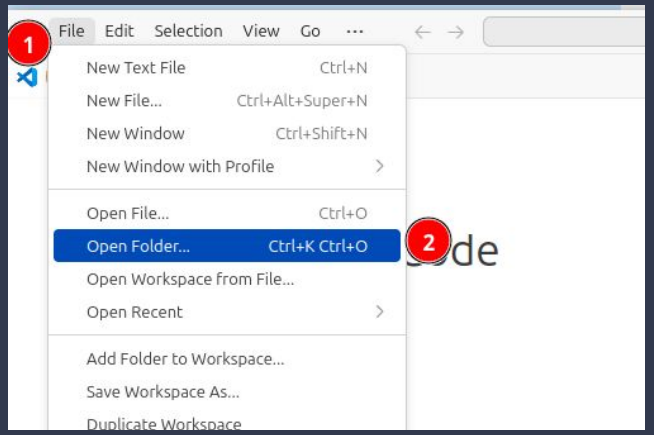
Llegamos a él con los siguientes pasos:

1. Inicio de windows
2. Buscar VsCode o Visual Studio Code

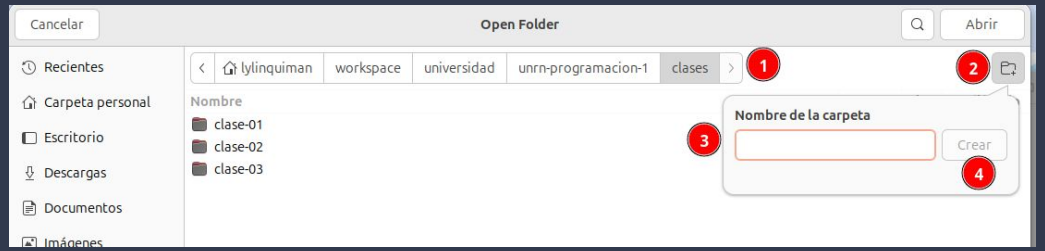


Repaso del entorno

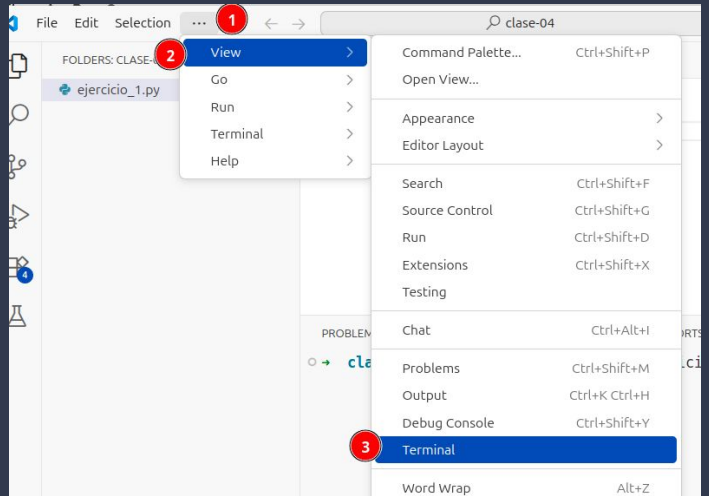
Preparando el ide para la clase



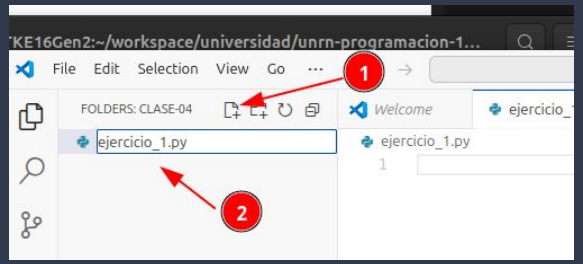
Crear carpeta con el nombre de clase y seleccionar



Abrimos la terminal

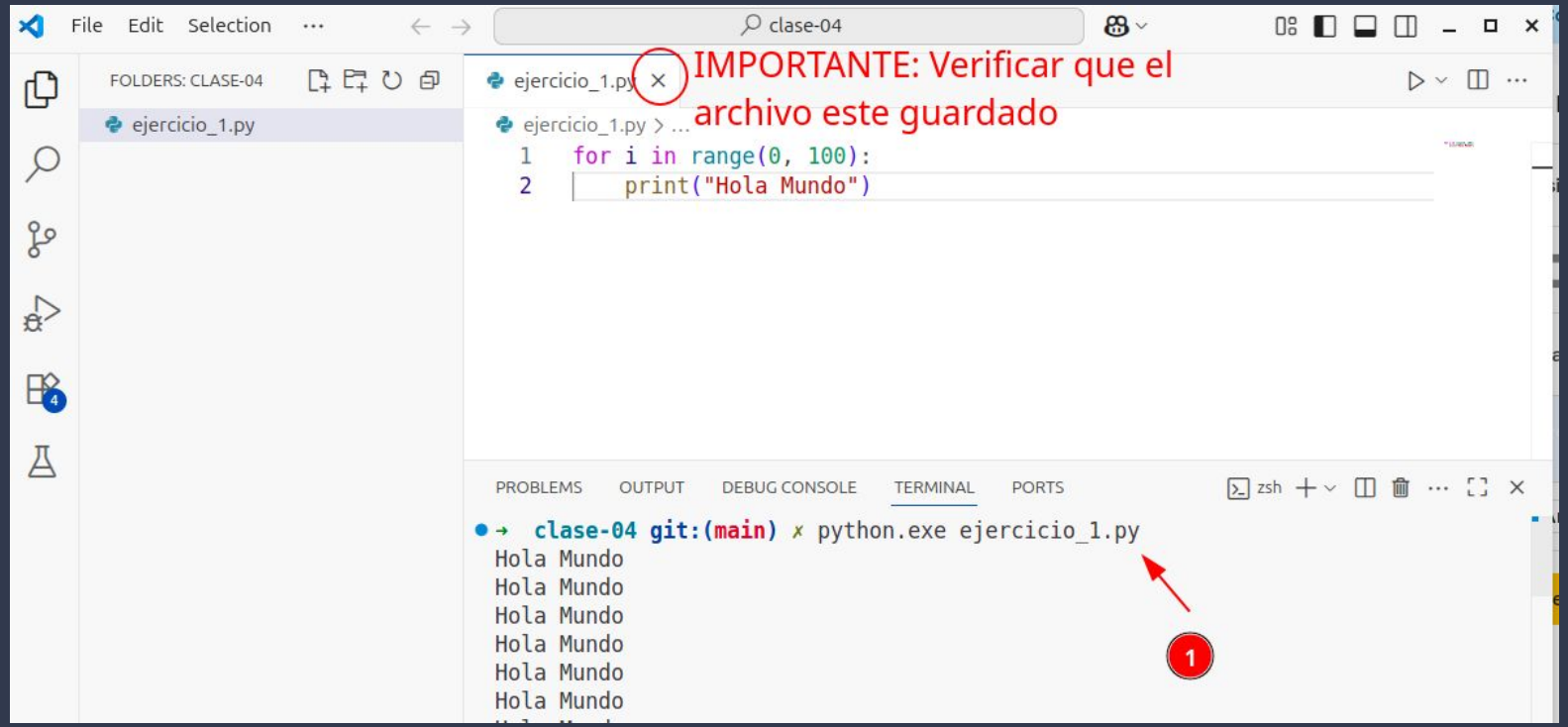


Creamos un archivo para cada ejercicio



Repaso del entorno

Preparando el ide para la clase parte 2



Repaso express de la clase anterior

- ¿Que nos puede devolver una condición lógica?
- ¿Podemos evaluar más de una condición al mismo tiempo?
- ¿Para qué sirve if, else y elif?
- ¿Para que sirve for?

WHILE

Repitiendo hasta el infinito y más allá

Iterador condicional: WHILE

El operador de control de flujo while nos permite repetir una acción mientras se cumpla una condición.

Si la condición es verdadera, el bloque de código se ejecuta.

Cuando la condición deja de cumplirse, el ciclo se detiene y el código continua.

La traducción de “while” es “mientras”.

Iterador condicional: WHILE - Ejemplo en el código

Pensemos en el ejemplo del acceso a un banco, en el acceso a un banco:

Mientras la contraseña ingresada sea incorrecta, el sistema seguirá pidiendo la contraseña.

Cuando la contraseña sea correcta, el sistema permitirá el acceso.

```
contraseña_registrada = "1234"
contraseña_correcta = False

while contraseña_correcta == False:
   contraseña_ingresada = input("Ingresar contraseña: ")

   if contraseña_ingresada == contraseña_registrada:
       contraseña_correcta = True

print("Acceso permitido")
```

```
contraseña_registrada = "1234"
contraseña_ingresada = ""

while contraseña_ingresada != contraseña_registrada:
   contraseña_ingresada = input("Ingresar contraseña: ")

print("Acceso permitido")
```

Iterador condicional: WHILE - Lupa

```
contraseña_registrada = "1234"
contraseña_correcta = False

while contraseña_correcta == False:
    contraseña_ingresada = input("Ingresar contraseña: ")

    if contraseña_ingresada == contraseña_registrada:
        contraseña_correcta = True

print("Acceso permitido")
```

Condición del iterador

Bloque de código que se repite.
Siempre entre tabs

Iterador condicional: WHILE - Consideraciones

1. Si la condición se cumple por siempre, nunca salimos de la repetición y el programa nunca termina.

   Prueben:

```
while True:
    print("Holaaaaaa")
```

2. Se pueden usar en los whiles todas las operaciones booleanas de la clase anterior.
3. Dentro de un while podemos usar if, else, elif y for.

Ejemplos de aplicación – ¿Que está pasando acá?

```
contador = 1

while contador <= 5:
    print(contador)
    contador = contador + 1
    input("Apreta enter para continuar")
```

```
numero = -1

while numero < 0:
    numero = int(input("Ingresar un número valido: "))

print("Número válido ingresado")
```

```
opcion = ""
print("Selecciona la comida para llevar" )
while opcion != "terminar pedido" :
    opcion = input("Escriba la comida que quieras agregar al pedido:
").lower()
    if opcion == "pizza":
        print("Excelente! Agregamos una pizza a tu pedido" )
    elif opcion == "terminar pedido" :
        print("Cerrando pedido..." )
    else:
        print(f"Lo siento, no tenemos  {opcion}, prueba con otra cosa" )
```

Agreguemos empanadas y hamburguesas.

Y indiquemos arriba el comando para salir.

LISTAS

¿Qué son las listas?

Hasta ahora veníamos trabajando con variables que solos nos permitían un solo valor a la vez.
Las listas son un tipo de variable nos permiten almacenar diferentes valores en una sola variable.

Pensemos una lista de compras en el super, en la vida real nosotros lo anotariamos de la siguiente manera:

1. Huevos
2. Pan
3. Coca
4. Azúcar
5. Leche
6. Fideos

En python lo podríamos anotar de la siguiente manera:

```
lista_de_super = [
    "Huevos",
    "Pan",
    "Coca",
    "Azúcar",
    "Leche",
    "Fideos"
]
```

Definamos la lista y revisemos de qué tipo es con type(lista_de_super)

Listas

- Nos sirven para evitar código repetitivo

```
nombre1 = "Juan"
nombre2 = "Ana"
nombre3 = "Pedro"
```

```
nombres = ["Juan", "Ana", "Pedro"]
```

- Nos permiten agrupar datos y partir de esto podemos
  a. Contarlos
  b. Agregar
  c. Eliminar
  d. Modificarlos
- En otros lenguajes de programación pueden ser llamadas Array o Vectores.

Lista - List

La variable de tipo lista nos permite almacenar diferentes valores una sola variable.

Contenido puede ser de tipo string, entero, flotante, otras listas, etc.

EJEMPLOS

```
lista = [] # Lista vacia
lista_textos = ["Ana", "Pedro"]
lista_numeros = [1, 2, 3]
lista_flotantes = [1.1, 2.2, 3.3]
```

OPERACIONES

Nombre	Operador	Ejemplo
Agregar	append(elemento)	`lista_textos.append("Juan")`
Eliminar	remove(elemento)	`lista_textos.remove("Ana")`
Acceso por índice	[n]	`lista_textos[0]`
Último Elemento	[-1]	`lista_textos[-1]`
Cantidad de elementos	len(lista)	`count(lista_textos)`

Jugando con la lista del super

```python
lista_super = []

# Agregar elementos
lista_super.append("huevos")
lista_super.append("pan")
lista_super.append("leche")

# Mostrar lista
print(lista_super)

# Eliminar un elemento
lista_super.remove("huevos")
```

```python
# Cantidad de elementos
print(len(lista_super))

# Primer elemento
print(lista_super[0])

# Último elemento
print(lista_super[-1])
```

Ejercicio: Crea tu propio anotador de compras

Usa un while y una lista.

- Usar input para preguntar que agregar a la lista
- Usar append para agregar los elementos a la lista.
- La lista de compra solo puede tener 5 cosas.
- Cuando termine imprimí la lista y la cantidad de cosas en pantalla.

PDFS y TAREAS

Todo el material en https://github.com/Ymil/unrn-programacion-1/

FIN

Volvemos el miércoles a las 17